IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

CRUD desde Python

sqlite3 y MongoDB sin salir del cuaderno

Sprint 5 · S05_06 · 30 min en clase (ejercicios 1 a 3)

CRUD

Cuatro operaciones, dos lenguajes

Operación	SQL	En Python (sqlite3 y pandas)	En MongoDB
Create	INSERT	`con.execute("INSERT...")`, `df.to_sql()`	`insert_one()`
Read	SELECT	`pd.read_sql_query()`	`find_one()`, `find()`
Update	UPDATE	`con.execute("UPDATE...")`	`update_one()` con `$set`
Delete	DELETE	`con.execute("DELETE...")`	`delete_many()`

Lo que hacías en SQL, ahora **desde un programa**: sin abrir ninguna herramienta gráfica.

LEER · EJERCICIO 1

Una consulta SQL, un DataFrame

Tabla de ejemplo **habitaciones**: (1, individual, 70) · (2, doble, 90) · (3, doble, 100) · (4, familiar, 130).

```
con = sqlite3.connect("hotel.db")
con.execute("SELECT COUNT(*) FROM habitaciones").fetchone()[0]   # 4
pd.read_sql_query("SELECT tipo, AVG(precio) AS precio_medio "
                  "FROM habitaciones GROUP BY tipo ORDER BY tipo", con)
```

tipo	precio_medio
doble	95.0
familiar	130.0
individual	70.0

CONSULTAS CON PARÁMETROS · EJERCICIO 2

Nunca pegues texto en el SQL

MAL

```
# MAL: pega el dato en el texto
dato = "0 OR 1=1"
con.execute("SELECT * FROM habitaciones "
            "WHERE id = " + dato)
```

Devuelve las 4 filas: el dato se ha colado como código SQL (inyección SQL).

BIEN

```
# BIEN: marcador ? y valores aparte
con.execute("SELECT * FROM habitaciones "
            "WHERE id = ?", (dato,))
```

Devuelve **0 filas**: el dato se trata siempre como un valor, nunca como código.

En el ejercicio 2: una función que usa ? y devuelve la lista de reservas de un cliente.

CREAR · EJERCICIO 3

Insertar filas y cargar un CSV

```
con.execute("INSERT INTO habitaciones VALUES (?, ?, ?)", (5, "doble", 105.0))
con.commit()
tarifas = pd.read_csv("tarifas.csv")
tarifas.to_sql("tarifas", con, if_exists="replace", index=False)
con.execute("SELECT COUNT(*) FROM tarifas").fetchone()[0]
```

commit()

Confirma los cambios: hasta entonces **nada es definitivo**.

to_sql

Carga un DataFrame entero en una tabla (**replace** la sustituye si existe).

Contar desde la base

Comprueba que **se guardó** y no solo que se leyó el CSV.

ACTUALIZAR Y BORRAR · EJERCICIO 4

Nada es definitivo hasta el commit

Tabla con 5 habitaciones (ids 1 a 5; las 3 dobles son 2, 3 y 5).

```
cur = con.execute("UPDATE habitaciones SET precio = 110 WHERE id = ?", (3,))
cur.rowcount                  # 1
con.commit()
cur = con.execute("DELETE FROM habitaciones WHERE tipo = 'doble'")
cur.rowcount                  # 3
con.rollback()                # deshecho: siguen las 5
```

rowcount

Cuántas filas tocaste. Si esperabas 1 y salen 6.000, **algo va mal**.

rollback

Deshace lo que no se ha confirmado con **commit**.

CONSULTAR PARA ANALIZAR · EJERCICIO 5

SUBSTR extrae un trozo del texto

fecha = '2025-03-14'	Resultado
SUBSTR(fecha, 1, 4)	'2025' (año)
SUBSTR(fecha, 6, 2)	'03' (mes)
SUBSTR(fecha, 9, 2)	'14' (día)

```
pd.read_sql_query("SELECT SUBSTR(fecha, 1, 4) AS anio, SUM(personas) AS total "
                  "FROM llegadas GROUP BY anio ORDER BY anio", con)
```

SUBSTR(texto, inicio, longitud): la posición de inicio **empieza en 1**. El resultado de la consulta llega como DataFrame, listo para analizar con pandas.

MONGODB · EJERCICIO 6

El mismo CRUD con documentos

Operación	En MongoDB (pymongo / mongomock)
Crear	`coleccion.insert_one({"_id": 1, "tipo": "doble", "precio": 90.0})`
Actualizar	`coleccion.update_one({"_id": 1}, {"$set": {"precio": 95.0}})`
Leer	`coleccion.find_one({"_id": 1})["precio"]`
Borrar	`coleccion.delete_many({"tipo": "doble"}).deleted_count`

Un documento es un diccionario de Python; **$set** cambia solo los campos indicados. **mongomock** imita MongoDB dentro del cuaderno, sin instalar nada.

HOY HAS APRENDIDO

Leer y escribir en una base desde Python

Parámetros

Siempre **?**, nunca texto pegado

Transacciones

commit confirma, **rollback** deshace

pandas

to_sql y **read_sql_query**

Esto es el paso 1 del reto: la serie diaria dentro de la base.